

Comunicación entre procesos mediante sockets TCP

Nicolle Bustamante Hernández
Ingeniería de Sistemas y Computación
Universidad Tecnológica de Pereira
Pereira, Colombia
n.bustamante@utp.edu.co

Resumen—Este informe documenta un laboratorio de comunicación entre procesos mediante sockets TCP, realizado sobre dos máquinas virtuales de Ubuntu Server en VirtualBox, conectadas por una red solo-anfitrión con IPs fijas. Sobre ese montaje se ejecutó un servidor de eco en Python y su cliente, y se observó el ciclo de vida completo de una conexión TCP: apertura activa y pasiva, intercambio de datos y cierre ordenado. Seis experimentos guiados permitieron verificar en la práctica conceptos como la cuádrupla que identifica cada conexión, la diferencia entre el socket de escucha y el socket de conexión, la semántica de `recv()` al devolver cero bytes, el hecho de que TCP es un flujo de bytes sin fronteras de mensaje, el estado `TIME_WAIT` y el diagnóstico de fallos por capas. Además se implementaron cinco retos, entre ellos un registro de conexiones, un protocolo con delimitadores de mensaje y un servidor de comandos. Las observaciones coinciden con lo descrito en el RFC 9293 y en la documentación de Python, y las dificultades encontradas se reportan junto con su resolución.

Index Terms—Sockets TCP, comunicación entre procesos, cliente-servidor, Python, VirtualBox, RFC 9293

I. OBJETIVOS

El laboratorio busca que el estudiante comprenda, desde la práctica, cómo dos procesos en máquinas distintas se comunican mediante sockets TCP. Los resultados de aprendizaje que se persiguen son los siguientes:

- **RA1.** Configurar una red privada entre dos máquinas virtuales y verificar la conectividad en la capa de red y de transporte.
- **RA2.** Implementar en Python los dos extremos de una comunicación TCP usando la API de sockets.
- **RA3.** Explicar el ciclo de vida de una conexión TCP: apertura pasiva, apertura activa, transferencia y cierre.
- **RA4.** Identificar una conexión por su cuádrupla (IP origen, puerto origen, IP destino, puerto destino) y justificar por qué el servidor necesita dos sockets.
- **RA5.** Demostrar experimentalmente que TCP entrega un flujo de bytes sin fronteras de mensaje, y argumentar las consecuencias de ello para el diseño de un protocolo de aplicación.

Estos objetivos se abordan de forma progresiva a lo largo del informe: el montaje cubre RA1, la configuración del servidor y el cliente cubre RA2 y RA3, y los experimentos de la Parte C profundizan en RA4 y RA5.

II. DESCRIPCIÓN DEL MONTAJE

II-A. Software y creación de las máquinas virtuales

El laboratorio se desarrolló sobre un anfitrión Windows, con **Oracle VirtualBox 7.2.20** y la imagen ISO de **Ubuntu 26.04.1 LTS**. Se crearon dos máquinas virtuales, VM-SERVIDOR y VM-CLIENTE, cada una con 2 GB de RAM, 15 GB de disco y 2 núcleos de CPU. Los dos núcleos no los pide la guía, pero hacen la instalación de Ubuntu Server notablemente más rápida.

Cada VM lleva dos adaptadores de red. El primero es NAT y da salida a internet, necesaria para `apt update`. El segundo es solo-anfitrión (`vboxnet0`), la red privada del laboratorio: conecta las dos VMs entre sí y con el anfitrión. Separarlos evita mezclar el tráfico del laboratorio con el de las actualizaciones.

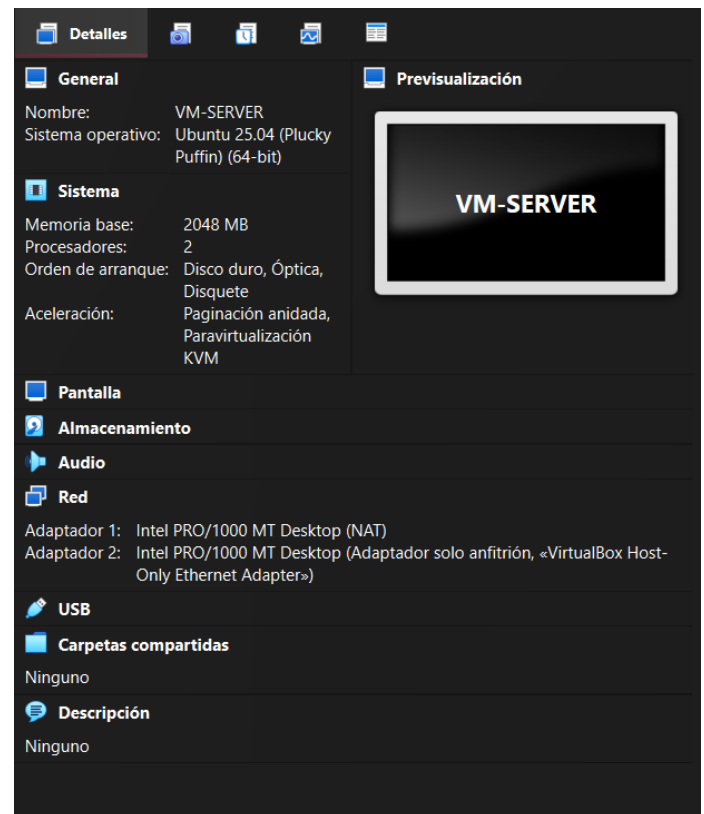


Figura 1. Configuración de la VM-SERVIDOR en VirtualBox: 2 GB de RAM, 2 núcleos de CPU, 15 GB de disco y los dos adaptadores de red (NAT y Host-only).

La instalación del sistema tardó unos veinte minutos y terminó sin problemas en la pantalla de login.

II-B. SSH y configuración de red

La guía menciona que el instalador permite marcar *OpenSSH server*, pero en esta instalación esa opción no apareció, así que SSH se instaló después con `apt install openssh-server` y se habilitó con `systemctl`. Hacerlo antes de configurar la red fue lo correcto: trabajar por SSH desde el CMD del anfitrión permite copiar, pegar, hacer scroll y mantener varias sesiones abiertas, algo que en la consola de VirtualBox es incómodo, sobre todo al editar archivos YAML.

Con `ip -br addr` se identificaron las interfaces: `enp0s3` (NAT, IP automática en `10.0.2.x`) y `enp0s8` (Host-only, la que llevaría IP fija). Se creó el archivo de `netplan`:

```
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
  addresses:
    - 192.168.56.10/24
```

La interfaz NAT queda en DHCP; la Host-only recibe la IP fija 192.168.56.10/24. Se ajustaron permisos con `sudo chmod 600 /etc/netplan/50-nat.yaml` (netplan exige que el archivo solo sea legible por su dueño) y se aplicó con `sudo netplan apply`.

```
socketlab@VM-SERVER:~$ ip -br addr
lo                UNKNOWN    127.0.0.1/8      ::1/128
enp0s3            UP          10.0.2.15/24     metric 100 fd17:625c:f037:2:a00:27ff:fe90:a1f6/64 fe00::a00:27ff:fe90:a1f6/64
enp0s8            UP          192.168.56.10/24 fe80::a00:27ff:fe7a:34e9/64
socketlab@VM-SERVER:~$
```

Figura 2. Salida de `ip -br addr` en la VM-SERVIDOR, mostrando la IP fija 192.168.56.10 en `enp0s8`.

II-C. Clonación y VM-CLIENTE

Con la VM-SERVIDOR apagada se usó la herramienta de clonación de VirtualBox, con clonación **completa y reiniciación de la MAC** de todas las tarjetas; si ambas VMs compartieran MAC, la red Host-only se confundiría. En la clon se cambió el hostname a `vm-cliente` y el mismo YAML de netplan, cambiando solo la IP a `192.168.56.11/24`.

```
socketlab@vm-cliente:~$ ip -br addr
lo                UNKNOWN    127.0.0.1/8      ::1/128
enp0s3            UP          10.0.2.15/24     metric 1024 fe17:a25c:f037:2:a00:27ff:fe12:a9cb/64 fe80::a00:27ff:fe12:a9cb/64
enp0s8            UP          192.168.56.11/24 fe80::a00:27ff:fe68:2728/64
```

Figura 3. Salida de `ip -br addr` en la VM-CLIENTE, mostrando la IP fija 192.168.56.11 en `enp0s8`.

II-D. Verificación de conectividad por capas

Con las IPs aplicadas, se verificó la conectividad capa por capa. Primero la de red, con `ping -c 4` en ambos sentidos: 4 respuestas y 0 % de pérdida en los dos casos, lo que confirma que las dos VMs están en la misma subred 192.168.56.0/24 y se ven.

[illegible]

Figura 4. Salida de `ping -c 4 192.168.56.1x` desde las VM, con 0 % de pérdida de paquetes.

Después se probó la capa de transporte con `nc -vz 192.168.56.10 5000`, que respondió `Connection refused`. Era el resultado esperado, porque todavía no había ningún servidor en el puerto 5000, pero el rechazo lo emite el kernel del servidor: la conexión llegó hasta la capa de transporte y la red funciona.

```
socketlab@vm-cliente:~$ nc -vz 192.168.56.10 5000
nc: connect to 192.168.56.10 port 5000 (tcp) failed: Connection refused
```

Figura 5. Salida de `nc -vz 192.168.56.10 5000` mostrando `Connection refused`, evidencia de que la capa de transporte responde aunque no haya servidor escuchando.

II-E. Curiosidades del montaje

Aparecieron dos comportamientos que, sin ser errores, conviene conocer. En la VM-SERVIDOR, a veces el arranque mostraba `vmwgfx seems to be running on an unsupported hypervisor`: es el driver gráfico del kernel quejándose del dispositivo virtual VMSVGA de VirtualBox, y no afecta al trabajo por terminal. En la VM-CLIENTE, el arranque a veces se demoraba en `systemd-networkd-wait-online.service` unos 42 segundos, porque ese servicio espera a que todas las interfaces tengan IP y la NAT depende de DHCP.

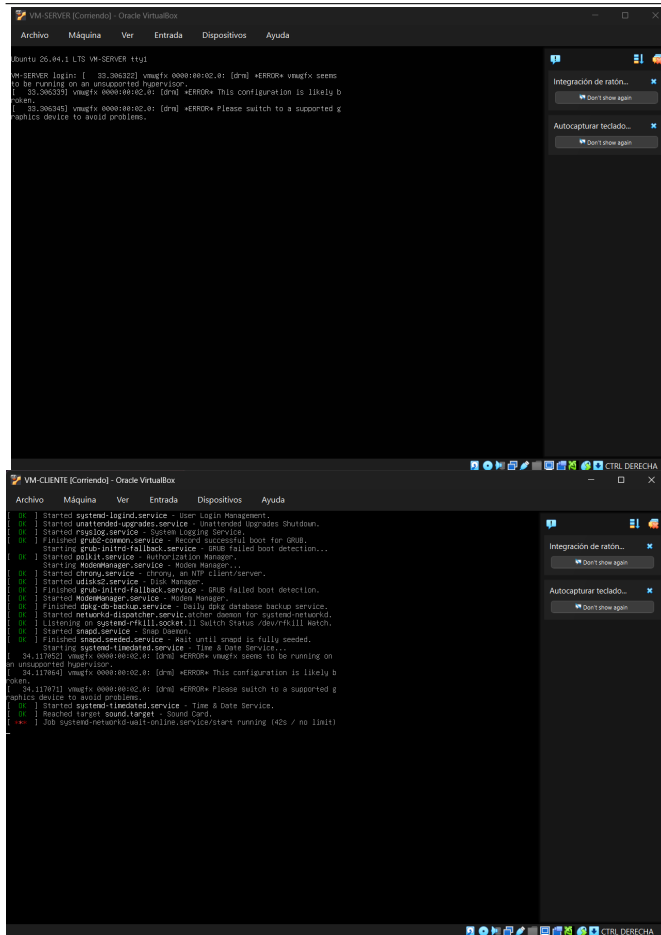


Figura 6. Comportamientos observados durante el arranque de las máquinas virtuales: mensaje relacionado con vmwgfx en la VM-SERVIDOR (izquierda) y demora del servicio systemd-networkd-wait-online.service en la VM-CLIENTE (derecha).

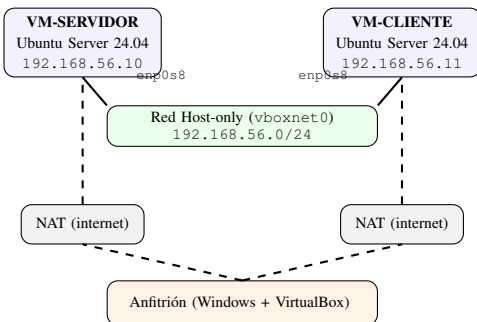


Figura 7. Diagrama de red del montaje. Las dos máquinas virtuales están conectadas por una red Host-only (vboxnet0) con IPs fijas en el rango 192.168.56.0/24. Cada VM tiene además un adaptador NAT que le da salida a internet, separado del tráfico del laboratorio.

III. PARTE B CONFIGURACIÓN DEL SERVIDOR Y PRIMER IDA Y VUELTA

Con la red funcionando, el siguiente paso fue escribir los dos extremos de la conversación TCP. La guía proporciona `servidor_eco.py`, `cliente_eco.py` y `cliente_rafaga.py`; en esta parte se usaron los dos

primeros. Los archivos se crearon con `nano` desde las sesiones SSH, uno en cada VM, pegando el contenido de la guía.

El servidor sigue el ciclo de vida clásico de una conexión TCP: crea el socket, lo asocia a una dirección con `bind()`, lo pone a escuchar con `listen()` y espera conexiones con `accept()`. El `bind()` usa `("0.0.0.0", 5000)`, donde la dirección comodín indica que el servidor acepta conexiones por cualquier interfaz de la máquina, y `listen(1)` define el tamaño de la cola de conexiones pendientes no el número máximo de clientes, que es un detalle que suele confundirse. `accept()` se bloquea hasta que llega un cliente y devuelve un socket nuevo dedicado a esa conexión. El servidor también activa `SO_REUSEADDR`, cuya razón de ser se estudia en E5.

El cliente es más corto: crea el socket del mismo tipo, se conecta con `connect()` y luego entra en un bucle donde lee texto del teclado, lo envía con `sendall()` y espera la respuesta con `recv()`, hasta que el usuario escribe `salir`. No hace `bind()`: el sistema operativo le asigna un *puerto efímero*, que el programa imprime con `getsockname()` para que se pueda ver la cuádrupla completa.

En la primera ejecución conjunta, el servidor quedó bloqueado en `accept()` tras imprimir:

```
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para
terminar)
```

Desde la VM-CLIENTE se lanzó `python3 cliente_eco.py 192.168.56.10 5000`, y el cliente respondió:

```
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] conectado
[cliente] extremo local : ('192.168.56.11', 34567)
<- puerto efímero asignado por el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
```

Entre bastidores ocurrió el saludo de tres vías, que no lo programa el estudiante: vive dentro de la pila TCP del sistema operativo y solo se manifiesta como el tiempo que tarda `connect()` en retornar. El puerto 34567 de la línea extremo local es el puerto efímero asignado, y cambia en cada ejecución.

```
socketlab@VM-SERVIDOR:~$ python3 servidor_eco.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

socketlab@vm-cliente:~$ python3 cliente_eco.py 192.168.56.10 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] conectado
[cliente] extremo local : ('192.168.56.11', 51564) <- pue
rto efímero asignado por el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
[cliente] escriba un texto y presione Enter. Escriba 'sal
ir' para terminar.
>
```

Figura 8. Ejecución del servidor y conexión del cliente: el servidor permanece escuchando en 0.0.0.0:5000 mediante `accept()`, mientras el cliente establece la conexión desde un puerto efímero hacia dicho puerto.

Al enviar `hola` desde el cliente, el servidor mostró la conexión aceptada con sus tres direcciones y los bytes en cada sentido, mientras el cliente mostró sus 4 bytes enviados y la respuesta recibida:

```
[servidor] recibidos 4 bytes: b'hola'
[servidor] enviados 4 bytes: b'HOLA'
```

```
[cliente] enviados 4 bytes: b'hola'
[cliente] recibidos 4 bytes: HOLA
```

Leídas lado a lado, las dos terminales cuentan el ida y vuelta completo: el mensaje viajó al servidor, se transformó en mayúsculas y regresó.

```
socketlab@VM-SERVER:~$ python3 servidor_eco.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

[servidor] --- conexión aceptada ---
[servidor] cliente remoto : 192.168.56.11:51564
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 51564)
[servidor] socket de escucha sigue siendo: ('0.0.0.0', 5000) (distinto objeto)
[servidor] recibidos 4 bytes: b'hola'
[servidor] enviados 4 bytes: b'HOLA'
[servidor] recibidos 2 bytes: b'ss'
[servidor] enviados 2 bytes: b'SS'
[servidor] recibidos 1 bytes: b'A'
[servidor] enviados 1 bytes: b'A'
[servidor] recibidos 1 bytes: b'A'
[servidor] recv() devolvió 0 bytes -> el cliente cerro la conexión
[servidor] conexión cerrada. Vuelvo a accept()

socketlab@VM-cliente:~$ python3 cliente_eco.py 192.168.56.10 5000
10 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] conectado
[cliente] extremo local : ('192.168.56.11', 51564) <- puerto efímero asignado por el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
[cliente] escriba un texto y presione Enter. Escriba 'salir' para terminar.
> hola
[cliente] enviados 4 bytes: b'hola'
[cliente] recibidos 4 bytes: HOLA
> ss
[cliente] enviados 2 bytes: b'ss'
[cliente] recibidos 2 bytes: SS
> A
[cliente] enviados 1 bytes: b'A'
[cliente] recibidos 1 bytes: A
> A
[cliente] enviados 1 bytes: b'A'
[cliente] recibidos 1 bytes: A
> salir
[cliente] socket cerrado
socketlab@VM-cliente:~$
```

Figura 9. Conexión TCP establecida entre el cliente y el servidor, mostrando en el servidor las direcciones de escucha, local y remota, junto con los bytes transmitidos en cada sentido, y el intercambio del mensaje hola → HOLA.

Al escribir salir, el cliente cerró su socket y el servidor imprimió recv() devolvió 0 bytes → el cliente cerró la conexión. Esa señal de cierre ordenado se analiza a fondo en E3.

Durante la ejecución se tomaron dos evidencias adicionales del estado del canal. Con `ss -ltnp` en la VM-SERVIDOR apareció el socket de escucha en estado LISTEN sobre 0.0.0.0:5000; con `ss -tn state established '( sport = :5000 )'` apareció la conexión establecida con su cuádrupla completa. La diferencia entre los dos comandos es clave y se retoma en E1 y E2: el primero muestra sockets que escuchan, el segundo muestra conexiones que existen. Un socket de escucha no es una conexión: es un socket que produce conexiones.

```
socketlab@VM-SERVER:~$ ss -ltnp
State Recv-Q Send-Q Local Address:Port Peer Address:Port
Process
LISTEN 0 1 0.0.0.0:5000 0.0.0.0:*
users:((("python3",pid=1618,fd=3))
LISTEN 0 4096 127.0.0.53:lo:53 0.0.0.0:*

LISTEN 0 4096 0.0.0.0:22 0.0.0.0:*

LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*

LISTEN 0 4096 [::]:22 [::]:*

socketlab@VM-SERVER:~$ ss -tn state established '( sport = :5000 )'
Recv-Q Send-Q Local Address:Port Peer Address:Port
0 0 192.168.56.10:5000 192.168.56.11:36158
```

Figura 10. Estado de los sockets TCP en la VM-SERVIDOR: `ss -ltnp` muestra el socket en estado LISTEN sobre 0.0.0.0:5000, mientras que la consulta de conexiones establecidas muestra la cuádrupla correspondiente a la conexión activa con el cliente.

IV. E1 LA CONEXIÓN TIENE CUATRO IDENTIFICADORES, NO UNO

IV-A. Predicción

Se esperaba que al conectar varios clientes al mismo puerto del servidor, cada conexión tuviera algún elemento distintivo. El servidor solo abre un puerto, el 5000, así que “algo” tenía que diferenciar una conexión de otra, aunque no estaba claro qué.

IV-B. Observación

Con el servidor corriendo, se lanzaron tres instancias de `cliente_eco.py` en tres terminales SSH de la VM-CLIENTE, todas contra 192.168.56.10 5000. Desde la VM-SERVIDOR se ejecutó `ss -tn state established '( sport = :5000 )'`, que muestra las conexiones establecidas cuyo puerto de origen es el 5000. La salida fue:

Recv-Q	Send-Q	Local Address:Port	Peer
0	0	192.168.56.10:5000	192.168.56.11:50486
0	0	192.168.56.10:5000	192.168.56.11:50494
0	0	192.168.56.10:5000	192.168.56.11:38412

```
socketlab@VM-SERVER:~$ ss -tn state established '( sport = :5000 )'
Recv-Q Send-Q Local Address:Port Peer Address:Port
0 0 192.168.56.10:5000 192.168.56.11:36158
0 0 192.168.56.10:5000 192.168.56.11:48748
0 0 192.168.56.10:5000 192.168.56.11:37648
```

Figura 11. Salida de `ss -tn state established '( sport = :5000 )'` en la VM-SERVIDOR, mostrando las tres conexiones con sus cuádruplas completas.

IV-C. Explicación

Cada fila es una conexión TCP completa, identificada por la cuádrupla (IP origen, puerto origen, IP destino, puerto destino). En las tres filas la IP y puerto del servidor son idénticos (192.168.56.10:5000), y la IP del cliente también (192.168.56.11), porque las tres conexiones vienen de la misma VM. Lo único que cambia es el puerto efímero del cliente: 50486, 50494 y 38412. Como cada cliente recibe un puerto efímero distinto, asignado al azar por el sistema operativo, cada conexión es única aunque el servidor use siempre el mismo puerto. Es lo que dice el RFC 9293 en su sección 3.4.1: una conexión queda definida por un par de sockets, y cada socket es la combinación de una dirección IP y un puerto.

IV-D. Respuestas a las preguntas

E1.1 ¿Qué valor es idéntico en las tres conexiones y cuál cambia?

Idénticos: la IP y puerto del servidor (192.168.56.10:5000) y la IP del cliente

(192.168.56.11). Cambia el puerto efímero del cliente (50486, 50494, 38412), asignado al azar a cada conexión. Esa diferencia es la que permite al servidor distinguir una conexión de otra.

E1.2 Si el servidor solo tiene un puerto 5000 abierto, ¿cómo logra el sistema operativo entregarle a cada cliente su propia respuesta y no la del vecino?

Porque el sistema operativo no identifica las conexiones por el puerto solo, sino por la cuádrupla completa. El kernel del servidor mantiene una tabla interna de conexiones y sabe qué bytes pertenecen a cada una. Cuando el servidor llama a `recv()` sobre el socket que devolvió `accept()`, el kernel le entrega los bytes de esa conexión específica, no los de las otras.

E1.3 ¿Cuántas conexiones simultáneas podría aceptar teóricamente este servidor desde un mismo cliente? ¿Qué recurso se agota primero?

Teóricamente muchísimas: el límite teórico es el rango de puertos efímeros, que en Linux va de 32768 a 60999, unos 28 000 puertos. En la práctica se agotan antes otras cosas: cada socket abierto consume un descriptor de archivo (unos 1024 por proceso por defecto, aunque es configurable) y cada conexión consume buffers del kernel. Los descriptores de archivo son el límite real, porque se agotan mucho antes que los puertos efímeros.

E1.4 El servidor tal como está escrito atiende a un cliente a la vez. ¿Los otros dos clientes reciben un error, o simplemente esperan? Justifique su respuesta con el papel de `listen(1)`.

Simplemente esperan. Cuando un cliente hace `connect()`, el kernel del servidor completa el saludo de tres vías y coloca la conexión en la cola de pendientes que define `listen(1)`. Mientras el servidor atiende a un cliente, las conexiones nuevas se acumulan ahí; con `listen(1)` la cola tiene capacidad para una conexión pendiente además de la que se está atendiendo, pero como el servidor llama a `accept()` apenas termina cada conexión, en la práctica la cola casi no se llena. Los clientes no reciben error: quedan esperando a que el servidor esté libre. Solo si la cola se llenara podría aparecer un error, y con un servidor que acepta rápido eso no ocurre. De nuevo, el argumento de `listen()` es el tamaño de la cola, no el número máximo de clientes.

V. E2 EL SERVIDOR MANEJA DOS SOCKETS DISTINTOS

V-A. Predicción

Se esperaba que el servidor imprimiera varias direcciones al aceptar una conexión y que fueran distintas entre sí. La intuición inicial era que un socket era un socket, y que el mismo objeto podía servir tanto para escuchar como para intercambiar datos.

V-B. Observación

Con el servidor corriendo y un cliente conectado, la salida al aceptar la conexión fue:

```
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para
terminar)

[servidor] ---- conexion aceptada ----
[servidor] cliente remoto : 192.168.56.11:51684
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 51684)
[servidor] socket de escucha sigue siendo:
('0.0.0.0', 5000) (distinto objeto)
```

Hay tres direcciones distintas: la del socket de escucha (0.0.0.0:5000), la del extremo local de la conexión (192.168.56.10:5000) y la del extremo remoto (192.168.56.11:51684). Además, el propio programa imprime que el socket de escucha sigue siendo el mismo objeto: no se transforma ni se reemplaza al aceptar.

```
socketlab@VM-SERVER:~$ python3 servidor_eco.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

[servidor] ---- conexion aceptada ----
[servidor] cliente remoto : 192.168.56.11:36158
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 36158)
[servidor] socket de escucha sigue siendo: ('0.0.0.0', 5000) (distinto objeto)
```

Figura 12. Salida del servidor al aceptar una conexión, mostrando las tres direcciones distintas: escucha, extremo local y extremo remoto.

```
socketlab@VM-SERVER:~$ ss -ltnp
State Recv-Q Send-Q Local Address:Port Peer Address:Port
Process
LISTEN 2 1 0.0.0.0:5000 0.0.0.0:*
users:(("python3",pid=1618,fd=3))
LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:*
LISTEN 0 4096 0.0.0.0:22 0.0.0.0:*
LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*
LISTEN 0 4096 [::]:22 [::]:*
```

Figura 13. Salida de `ss -ltnp` en la VM-SERVIDOR, mostrando el socket de escucha sobre 0.0.0.0:5000.

V-C. Explicación

El servidor maneja dos sockets distintos. El socket de escucha existe desde el arranque, su única función es aceptar conexiones y permanece siempre igual; su dirección es ('0.0.0.0', 5000) porque se configuró así en el `bind()`: la dirección comodín significa "acepto conexiones por cualquier interfaz de esta máquina". El socket de la conexión, devuelto por `accept()`, es un objeto nuevo dedicado a esa conexión específica: su extremo local es ('192.168.56.10', 5000) porque el kernel lo asocia a la IP concreta por la que entró la conexión (la interfaz Host-only), y su extremo remoto es la IP y puerto efímero del cliente. El socket de escucha no aparece en ninguna operación de datos: todos los `recv()` y `sendall()` se hacen sobre el socket de conexión.

V-D. Respuestas a las preguntas

E2.1 ¿Por qué el socket de escucha muestra 0.0.0.0 y el socket de la conexión muestra 192.168.56.10?

Porque el de escucha se configuró con la dirección comodín, que acepta conexiones por cualquier interfaz (NAT, Host-only o loopback), mientras que el socket de conexión queda atado a la interfaz concreta por la que llegó esa conexión. El cliente se conectó desde 192.168.56.11, su paquete entró por la interfaz Host-only del servidor, y el kernel fijó esa IP en el socket resultante para que las respuestas salgan por la misma interfaz por la que llegó la petición.

E2.2 ¿Qué pasaría si el servidor intentara hacer `recv()` sobre el socket de escucha en lugar del socket devuelto por `accept()`?

El `recv()` se bloquearía indefinidamente o lanzaría un error, según el sistema, porque el socket de escucha es un socket pasivo sin buffer de datos de ninguna conexión. Además, los datos del cliente se perderían de esa perspectiva: el kernel ya los entregó al socket de conexión cuando llegaron, y el socket de escucha nunca los ve.

E2.3 Relación lo observado con esta afirmación de la documentación de Python: “Esto es todo lo que hace un socket servidor. No envía datos. No recibe datos. Solo produce sockets cliente.”

La salida observada la demuestra punto por punto. El socket de escucha se creó, se asoció a una dirección y se marcó como pasivo; produjo un socket nuevo con `accept()`; y no aparece en ninguna operación de datos, mientras el socket de escucha sigue existiendo con la misma dirección ('0.0.0.0', 5000), como confirma la línea `socket de escucha sigue siendo....` Esa es exactamente la razón por la que el servidor necesita dos sockets: uno para escuchar y otro, por cada conexión, para intercambiar datos.

VI. E3 QUÉ SIGNIFICA QUE RECV() DEVUELVA CERO

VI-A. Predicción

Se esperaba que al cerrar el cliente, el servidor terminara la conexión de alguna manera. La duda era cómo distinguiría “el cliente cerró” de “el cliente está callado pero sigue conectado”: desde el socket, en ambos casos simplemente no hay datos que leer.

VI-B. Observación

Se arrancó el servidor, se conectó un cliente, se intercambiaron varios mensajes (hola, mundo) y se cerró el cliente con salir. El cliente imprimió [cliente] socket cerrado y el servidor:

```
[servidor] recv() devolvio 0 bytes -> el cliente
           cerro la conexion
[servidor] conexion cerrada. Vuelvo a accept()
```

Luego se comentó la línea `if not datos: break` del servidor y se repitió la prueba. Esta vez, al cerrar el cliente, el servidor entró en un bucle infinito:

```
[servidor] recibidos 0 bytes: b''
[servidor] enviados 0 bytes: b''
[servidor] recibidos 0 bytes: b''
...
```

La terminal se llenó de mensajes a gran velocidad y el proceso consumió el 100 % de un núcleo de CPU. Hubo que detenerlo con `Ctrl+C` repetidamente o cerrar la sesión.

[illegible]

Figura 14. Bucle infinito del servidor al comentar la comprobación `if not datos: break`, mostrando la pantalla llena de recibidos 0 bytes: `b''`.

VI-C. Explicación

`recv()` tiene dos comportamientos distintos. Si no hay datos pero el otro extremo sigue conectado, se bloquea: el programa espera. Si el otro extremo cerró, devuelve `b` inmediatamente, y lo hace una y otra vez en cada llamada, porque la conexión ya está cerrada. La documentación de Python lo dice así: cuando `recv` devuelve 0 bytes, el otro lado ha cerrado o está en proceso de cerrar, y no se recibirán más datos en esa conexión, nunca.

El bucle infinito de la variante sale de ahí: sin la comprobación `if not datos: break`, cada `recv()` devuelve `b` al instante, el servidor imprime el mensaje, calcula `b.upper()` (que sigue siendo `b`), “envía” `b` y vuelve al inicio. Como nada se bloquea, el bucle gira a velocidad máxima y satura la CPU y la terminal. La conclusión es directa: esa comprobación no es opcional; es la única forma que tiene el servidor de enterarse del cierre y de volver a `accept()` para atender a otros clientes.

VI-D. Respuestas a las preguntas

E3.1 ¿Qué imprime el servidor en el instante del cierre?

[servidor] recv() devolvio 0 bytes ->el cliente cerro la conexion y [servidor] conexion cerrada. Vuelvo a accept(). Aparece en el momento exacto en que el cliente ejecuta close() (o presiona Ctrl+C, que internamente cierra el socket). El kernel del cliente envía un segmento FIN, el kernel del servidor lo recibe, marca la conexión como semi-cerrada y hace que la siguiente llamada a recv() devuelva b.

E3.2 ¿Cuál es la diferencia entre “recv() devolvió 0 bytes” y “recv() está esperando y todavía no hay datos”? ¿Cómo distingue el programa una situación de la otra?

La diferencia está en el valor de retorno frente al bloqueo. Si `recv()` no ha retornado todavía, el programa está bloqueado esperando datos, y todavía no sabe si van a llegar: el “no hay nada” se manifiesta como bloqueo, no como valor. Si retorna b

, el otro extremo cerró, y es una señal inequívoca que se repite en cada llamada. `recv()` siempre devuelve bytes, vacíos o no. Por eso `if not datos: break` funciona: `not b` es `True` y se ejecuta el `break`, mientras que `not b"hola"` es `False` y el bucle continúa.

E3.3 Modifique el servidor eliminando la comprobación `if not datos: break` y vuelva a ejecutar la prueba. Describa lo que ocurre y explique por qué ocurre.

Ocorre lo observado: el servidor entra en un bucle infinito, imprimiendo recibidos 0 bytes: `b''` sin parar y consumiendo el 100 % de un núcleo. Ocurre porque, con la conexión ya cerrada, `recv()` devuelve `b''` inmediatamente en lugar de bloquearse, y el `while True` ya no tiene condición de salida. El servidor queda atrapado en esa conexión, gastando CPU y sin poder aceptar nuevas conexiones. Es un error clásico de programación de servidores TCP.

E3.4 ¿Qué pasaría si, en lugar de cerrar el cliente ordenadamente, se apagara de golpe la VM-CLIENTE? ¿El servidor se enteraría de inmediato? ¿Por qué?

Depende de cómo se apague. Con un apagado ordenado (sudo poweroff), el sistema operativo cierra todos los sockets antes de apagarse, envía el FIN, y el servidor se entera de inmediato o casi: `recv()` devuelve `b''` como en el caso normal.

Con un apagado abrupto (cerrar VirtualBox de golpe o apagar el anfitrión) no se envía ningún FIN, porque el kernel se apaga antes de poder enviar nada. El servidor no se entera: sigue bloqueado en `recv()`, pensando que el cliente está vivo. Solo lo descubre cuando TCP detecta que el otro extremo no responde, y eso puede tardar mucho: si el servidor intenta enviar datos y no recibe ACK, retransmite con backoff exponencial durante varios minutos antes de declarar la conexión muerta; y si solo escucha, sin keepalive activado (que no es el caso por defecto), puede pasar del orden de un cuarto de hora o más sin que nada lo alerte. Esta es la razón por la que en sistemas distribuidos reales se usan timeouts explícitos y keepalives: no se puede confiar en que el otro extremo siempre cierre ordenadamente.

VII. E4 TCP ES UN FLUJO DE BYTES

VII-A. Predicción

Se esperaba que el servidor recibiera uno, dos y tres en el mismo orden y con sus fronteras conservadas: un `sendall()` por cada `recv()` con exactamente los mismos bytes. Como se verá, esa era justo la parte que TCP nunca prometió.

VII-B. Observación

Se ejecutó el cliente ráfaga dos veces, variando la pausa entre envíos. Sin pausa (`python3 cliente_rafaga.py 192.168.56.10 5000 0`), el servidor hizo dos `recv()` en lugar de tres:

```
[servidor] recibidos 3 bytes: b'uno'
[servidor] enviados 3 bytes: b'UNO'
[servidor] recibidos 7 bytes: b'dostres'
[servidor] enviados 7 bytes: b'DOSTRES'
```

Con pausa de 0.5 segundos, el servidor hizo tres `recv()` separados (uno, dos, tres), pero el cliente, con una sola llamada a `recv()`, recibió los tres ecos pegados:

```
[rafaga] recv <- b'UNODOSTRES' (10 bytes)
```

Es decir: sin pausa, dos `recv()` del servidor; con pausa, tres `recv()` del servidor pero un solo `recv()` del cliente con los 10 bytes juntos.

```

[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

[servidor] --- conexión aceptada ---
[servidor] cliente remoto : 192.168.56.11:55718
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 55718)
[servidor] socket de escucha sigue siendo: ('0.0.0.0', 5000) (distinto objeto)
[servidor] recibidos 10 bytes: b'unodostres'
[servidor] enviados 10 bytes: b'UNODOSTRES'
[servidor] recv() devolvió 0 bytes -> el cliente cerró la conexión
[servidor] conexión cerrada. Vuelvo a accept()

[servidor] --- conexión aceptada ---
[servidor] cliente remoto : 192.168.56.11:55722
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 55722)
[servidor] socket de escucha sigue siendo: ('0.0.0.0', 5000) (distinto objeto)
[servidor] recibidos 3 bytes: b'uno'
[servidor] enviados 3 bytes: b'UNO'
[servidor] recibidos 3 bytes: b'dos'
[servidor] enviados 3 bytes: b'DOS'
[servidor] recibidos 4 bytes: b'tres'
[servidor] enviados 4 bytes: b'TRES'
[servidor] recv() devolvió 0 bytes -> el cliente cerró la conexión

[socket:abre_cliente]-$ python3 cliente_rafaga.py 192.168.56.10 5000 0
[rafaga] conectado desde ('192.168.56.11', 55718) hacia ('192.168.56.10', 5000)
[rafaga] pausa entre envíos: 0.0 s
[rafaga] send -> b'uno' (3 bytes)
[rafaga] send -> b'dos' (3 bytes)
[rafaga] send -> b'tres' (4 bytes)
[rafaga] se enviaron 10 bytes en 3 llamadas a sendall()
[rafaga] ahora UNA sola llamada a recv()...
[rafaga] recv <- b'UNODOSTRES' (10 bytes)

[socket:abre_cliente]-$ python3 cliente_rafaga.py 192.168.56.10 5000 0.5
[rafaga] conectado desde ('192.168.56.11', 55722) hacia ('192.168.56.10', 5000)
[rafaga] pausa entre envíos: 0.5 s
[rafaga] send -> b'uno' (3 bytes)
[rafaga] send -> b'dos' (3 bytes)
[rafaga] send -> b'tres' (4 bytes)
[rafaga] se enviaron 10 bytes en 3 llamadas a sendall()
[rafaga] ahora UNA sola llamada a recv()...
[rafaga] recv <- b'UNODOSTRES' (10 bytes)

```

Figura 15. Comparación del comportamiento de TCP ante diferentes pausas entre envíos. Con pausa de 0 segundos, los tres `sendall()` del cliente fueron recibidos por el servidor mediante dos llamadas a `recv()`, mientras que con una pausa de 0.5 segundos el servidor realizó tres `recv()` separados. En ambos casos, el cliente recibió los datos de respuesta como un flujo de bytes, evidenciando que TCP no conserva los límites de los envíos realizados mediante `sendall()`.

VII-C. Explicación

El RFC 9293 define, en su sección 2.2.1, que TCP provee a las aplicaciones un servicio *fiabile, en orden, de flujo de bytes*. El experimento muestra exactamente qué promete cada palabra y qué no. Los 10 bytes llegaron completos (fiabile) y en el orden correcto (en orden); pero llegaron como un flujo continuo, y las fronteras entre los tres envíos se perdieron: dos y tres se entregaron pegados como `dostres`, sin que el servidor supiera que originalmente eran dos mensajes.

La razón es que TCP opera sobre buffers del kernel. El emisor escribe bytes en el buffer de envío y el receptor los lee del buffer de recepción; TCP decide internamente cómo agrupar y dividir esos bytes en segmentos y cuándo entregarlos, según el timing, el tamaño de los buffers y el sistema operativo. El programador no tiene control sobre eso, y la documentación de Python lo advierte: `send` y `recv` operan sobre los búferes de red y no necesariamente manejan todos los bytes que se les entrega; es responsabilidad del programador llamarlos de nuevo hasta que el mensaje esté completo.

La consecuencia de diseño es que la capa de aplicación no puede asumir que un `send()` corresponde a un `recv()`, y debe agregar sus propias fronteras: delimitadores, prefijos de longitud, mensajes de tamaño fijo o cierre de la conexión. Ese es precisamente el reto R2.

VII-D. Respuestas a las preguntas

E4.1 Tres llamadas a `sendall()` produjeron, en un caso, dos llamadas a `recv()` en el servidor. ¿Se perdió

algún byte? ¿Se desordenó alguno? Entonces, ¿qué fue exactamente lo que no se conservó?

No se perdió ningún byte (3 más 7 son los 10 enviados) ni se desordenó nada (uno llegó antes que dos, que llegó antes que tres). Lo que no se conservó fue la frontera entre dos y tres: se agruparon en una sola entrega, `dostres`, borrando la asociación uno a uno entre envíos y recepciones.

E4.2 En el caso con pausa de 0.5 s, el cliente hizo un solo `recv()` y obtuvo los tres ecos pegados. ¿Dónde estuvieron esos bytes mientras el cliente no los pedía?

En el buffer de recepción del kernel del cliente. El kernel fue guardando ahí cada eco conforme llegaba, y cuando el cliente finalmente llamó a `recv()`, el kernel le entregó todo lo acumulado: `UNODOSTRES`. `recv()` no pide datos a la red; lee del buffer local, y si hay varios mensajes acumulados los devuelve juntos, hasta el tamaño máximo especificado.

E4.3 Ejecute el experimento cinco veces seguidas con pausa 0. ¿Obtiene siempre el mismo resultado? ¿Qué implica esto para depurar un sistema distribuido?

No. Cada ejecución puede agrupar distinto: a veces uno más `dostres`, a veces `unodos` más `tres`, a veces los tres juntos o los tres separados. Depende del timing, de la carga del sistema y de los buffers del kernel. La implicación es que estos errores son no deterministas: el mismo código puede funcionar cien veces y fallar en la ciento uno. Cuando un sistema distribuido falla de forma intermitente, es muy probable que sea por fronteras de mensajes o condiciones de carrera, no por un bug lógico. Por eso los protocolos de aplicación deben delimitar explícitamente sus mensajes.

E4.4 Lea de nuevo la definición del RFC 9293: “TCP provee a las aplicaciones un servicio fiable, en orden, de flujo de bytes.” Señale qué parte de esa frase explica lo que se observó, y qué parte de la frase nunca prometió lo que se esperaba.

Fiable explica que no se perdió ningún byte; *en orden*, que llegaron en el orden correcto; y *flujo de bytes*, que llegaron como un flujo continuo sin fronteras. Lo que la frase nunca prometió es la correspondencia uno a uno entre un `send()` del emisor y un `recv()` del receptor con los mismos bytes. Esa “ilusión de mensajes” no la ofrece TCP, y el programador debe construirla por su cuenta.

E4.5 (Pregunta de transferencia.) Suponga que en lugar de uno, dos, tres estuviera enviando tres transferencias bancarias. ¿Qué tendría que agregarle al mensaje para que el receptor pudiera saber dónde termina una transferencia y empieza la siguiente? La documentación de Python menciona cuatro estrategias posibles: enúncielas y escoja una, justificando por qué.

Las cuatro estrategias son: mensajes de longitud fija (todos del mismo tamaño; inflexible y desperdicia espacio con datos variables); delimitadores (cada mensaje termina con un carácter especial, como `\n`; frágil si el delimitador puede aparecer dentro del contenido); prefijo de longitud (el mensaje empieza con un campo que dice cuántos bytes mide; el receptor lee el prefijo y luego exactamente esa cantidad); y cerrar la conexión (solo sirve para un mensaje por conexión, lo cual es costoso).

Para transferencias bancarias, la elección es el prefijo de longitud. Las transferencias tienen tamaño variable (según monto, destinatario y metadatos), así que la longitud fija no aplica; el contenido puede ser binario o contener cualquier byte, incluido `\n`, así que los delimitadores son frágiles; y abrir y cerrar una conexión TCP por cada transferencia es ineficiente. El prefijo de longitud funciona con cualquier contenido y tamaño. Un protocolo concreto sería: cuatro bytes de longitud en big-endian seguidos de los datos; el receptor lee primero los cuatro bytes, calcula la longitud y lee exactamente esa cantidad. Sin importar cómo TCP agrupe los bytes, el receptor siempre puede reconstruir los mensajes originales.

E4.6 El protocolo HTTP resuelve este mismo problema con la cabecera `Content-Length`. ¿A cuál de las cuatro estrategias del punto anterior corresponde?

Al prefijo de longitud. `Content-Length` es exactamente eso: un campo en la cabecera que dice cuántos bytes mide el cuerpo; el receptor lee las cabeceras, extrae el valor, y lee exactamente esa cantidad. HTTP también tiene `Transfer-Encoding: chunked`, una variante en la que cada chunk va precedido por su propia longitud y la secuencia termina con un chunk de longitud 0; es útil cuando no se conoce la longitud total de antemano.

VIII. E5 EL ESTADO `TIME_WAIT` Y LA OPCIÓN `SO_REUSEADDR`

VIII-A. Predicción

Se esperaba que reiniciar el servidor inmediatamente después de cerrarlo produjera algún error, aunque no estaba claro si vendría de Python o del sistema operativo, ni si sería inmediato.

VIII-B. Observación

Se comentó la línea de `SO_REUSEADDR` en `servidor_eco.py`, se arrancó el servidor, se conectó un cliente, se enviaron mensajes y se cerró el cliente. Al detener el servidor con `Ctrl+C` e intentar arrancarlo de nuevo, apareció:

```
Traceback (most recent call last):
  File "/home/socketlab/servidor_eco.py", line 87,
    in <module>
    main()
  File "/home/socketlab/servidor_eco.py", line 33,
    in main
    servidor.bind((HOST, PUERTO))
OSError: [Errno 98] Address already in use
```

Después se buscó el estado `TIME_WAIT` con `ss -tan state time-wait` en ambas VMs. En el cliente aparecieron conexiones en `TIME_WAIT` apuntando al servidor:

Recv-Q	Send-Q	Local Address:Port	Peer Address:
		Port	
0	0	192.168.56.11:41664	
		192.168.56.10:5000	
0	0	192.168.56.11:51090	
		192.168.56.10:5000	
0	0	192.168.56.11:53684	
		192.168.56.10:5000	

Al invertir el sentido del cierre, haciendo que el servidor terminara primero la conexión, el estado apareció esta vez en la VM-SERVIDOR. Eso confirmó experimentalmente que el estado `TIME_WAIT` pertenece al extremo que realiza el cierre activo.

```
socketlab@VM-SERVER:~$ python3 servidor_eco.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

[servidor] ---- conexion aceptada ----
[servidor] cliente remoto : 192.168.56.11:57112
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 57112)
[servidor] socket de escucha sigue siendo: ('0.0.0.0', 5000) (distinto objeto)
^C
[servidor] interrumpido por el usuario
[servidor] socket de escucha cerrado
socketlab@VM-SERVER:~$ python3 servidor_eco.py
Traceback (most recent call last):
  File "/home/socketlab/servidor_eco.py", line 87, in <module>
    main()
    ~~~~^~
  File "/home/socketlab/servidor_eco.py", line 33, in main
    servidor.bind((HOST, PUERTO))
    ~~~~~^~
OSError: [Errno 98] Address already in use
```

Figura 16. Error `OSError: [Errno 98] Address already in use` al intentar reiniciar el servidor inmediatamente después de cerrarlo.

```
socketlab@VM-SERVER:~$ python3 servidor_eco.py
Traceback (most recent call last):
  File "/home/socketlab/servidor_eco.py", line 87, in <module>
    main()
    ~~~~^~
  File "/home/socketlab/servidor_eco.py", line 33, in main
    servidor.bind((HOST, PUERTO))
    ~~~~~^~
OSError: [Errno 98] Address already in use
socketlab@VM-SERVER:~$ ss -tan state time-wait
Recv-Q Send-Q Local Address:Port Peer Address:Port
0 0 192.168.56.10:5000 192.168.56.11:57112
```

Figura 17. Estado `TIME_WAIT` observado mediante `ss -tan state time-wait`. El estado aparece en el extremo que inicia el cierre activo de la conexión; en este caso, las conexiones mostradas corresponden a la VM-SERVER.

VIII-C. Explicación

`TIME_WAIT` es el estado en el que entra el extremo que realiza el cierre activo de la conexión. En las pruebas, cuando el cliente cerraba primero con `salir` o `Ctrl+C`, el estado aparecía en la VM-CLIENTE; al invertir el cierre, aparecía en la VM-SERVIDOR.

¿Por qué TCP impone esa espera en lugar de liberar el puerto de inmediato? Porque una conexión se identifica por la cuádrupla (sección E1). Si el puerto se reutilizara de inmediato, un segmento rezagado de la conexión anterior que puede vivir en la red hasta el MSL podría llegar después de abrirse una conexión nueva con la misma cuádrupla y ser interpretado como parte de ella, corrompiendo los datos. `TIME_WAIT` reserva

la cuádrupla el tiempo suficiente para garantizar que todo segmento rezagado ya haya muerto.

`SO_REUSEADDR` relaja esa restricción: permite el `bind()` sobre un puerto que aún tiene conexiones en `TIME_WAIT`. Introduce un riesgo teórico (el del segmento rezagado), pero en la práctica es mínimo porque TCP valida los números de secuencia y descarta segmentos con números antiguos. Por eso prácticamente todos los servidores de producción la activan: sin ella, cada reinicio implicaría esperar hasta dos minutos fuera de servicio, algo inviable en sistemas de orquestación modernos que reinician contenedores cada pocos segundos.

VIII-D. Respuestas a las preguntas

E5.1 ¿Qué excepción lanza Python al intentar el `bind()`?

`OSError: [Errno 98] Address already in use`. El código 98 corresponde a `EADDRINUSE` en Linux: el kernel se niega a asociar el socket a un puerto con conexiones recientes cuyo plazo aún no ha expirado.

E5.2 ¿Cuánto tiempo hay que esperar para que el puerto vuelva a estar disponible?

Unos 120 segundos: dos veces el MSL de 60 segundos, que es el valor por defecto en Linux. Es configurable a nivel de kernel. Mientras se espera, se puede observar cómo las conexiones en `TIME_WAIT` se extinguen con `watch -n 1 'ss -tan state time-wait'`.

E5.3 ¿Por qué el diseño de TCP impone ese estado de espera en lugar de liberar el puerto inmediatamente?

Por lo explicado arriba: si la cuádrupla se reutilizara de inmediato, un segmento rezagado de la conexión anterior podría confundirse con parte de la nueva. `TIME_WAIT` reserva la cuádrupla durante dos veces el MSL para garantizar que cualquier segmento rezagado haya muerto antes de la reutilización.

E5.4 ¿Qué riesgo introduce entonces `SO_REUSEADDR`? ¿Por qué, a pesar de ese riesgo, prácticamente todos los servidores de producción la activan?

El riesgo es que un segmento rezagado de una conexión anterior llegue a una conexión nueva que reusa la misma cuádrupla. Es un riesgo teórico y mínimo, porque TCP valida los números de secuencia y descarta los segmentos antiguos. A cambio, la ganancia operativa es grande: sin `SO_REUSEADDR` cada reinicio implica hasta dos minutos fuera de servicio, lo cual es inviable para servidores que se actualizan con frecuencia o para orquestadores como Kubernetes que reinician contenedores constantemente.

IX. E6 DIAGNÓSTICO POR CAPAS: PROVOCAR FALLOS A PROPÓSITO

IX-A. Predicción

Se esperaba que cada fallo produjera un error distinto y que esos errores permitieran identificar la capa del problema, subiendo desde la red hasta la aplicación. La idea era que los errores de red fueran lentos y los de transporte rápidos, aunque no estaba claro cuál sería cuál.

IX-B. Observación

Se provocaron cinco fallos, lanzando en cada caso el cliente y registrando el error exacto.

En F1 (servidor detenido) y F2 (puerto equivocado, 5001), el cliente respondió de inmediato: ERROR: conexion rechazada. En F3, contra la IP inexistente 192.168.56.99, respondió ERROR de red: [Errno 113] No route to host. En F4, con el servidor corriendo pero configurado con HOST = "127.0.0.1", el cliente desde la VM-CLIENTE obtuvo de nuevo conexion rechazada, aunque el proceso estaba vivo en su terminal. En F5, tras activar el cortafuegos con `sudo ufw enable`, el cliente esperó un rato y falló con ERROR de red: [Errno 110] Connection timed out; el servidor seguía corriendo y escuchando.

```
socketlab@vm-cliente:~$ python3 cliente_eco.py 192.168.56
.10 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] ERROR: conexion rechazada. Nadie escucha en 192.
168.56.10:5000
[cliente] socket cerrado
```

Figura 18. F1: Connection refused al lanzar el cliente con el servidor detenido.

```
socketlab@VM-SERVER:~$ python3 servidor_eco.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

socketlab@vm-cliente:~$ python3 cliente_eco.py 192.168.56
.99 5000
[cliente] conectando a 192.168.56.10:5001 ...
[cliente] ERROR: conexion rechazada. Nadie escucha en 192.
168.56.10:5001
[cliente] socket cerrado
socketlab@vm-cliente:~$
```

Figura 19. F2: Connection refused al lanzar el cliente contra un puerto equivocado.

```
socketlab@VM-SERVER:~$ python3 servidor_eco.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

socketlab@vm-cliente:~$ python3 cliente_eco.py 192.168.56
.10 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] ERROR de red: [Errno 113] No route to host
[cliente] socket cerrado
```

Figura 20. F3: [Errno 113] No route to host al lanzar el cliente contra una IP inexistente en la subred.

```
socketlab@VM-SERVER:~$ python3 servidor_eco.py
[servidor] socket de escucha en ('127.0.0.1', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

socketlab@vm-cliente:~$ python3 cliente_eco.py 192.168.56
.10 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] ERROR: conexion rechazada. Nadie escucha en 192.
168.56.10:5000
[cliente] socket cerrado
```

Figura 21. F4: Connection refused desde la VM-CLIENTE con el servidor corriendo solo en 127.0.0.1.

IX-C. Explicación

F1, F2 y F4 dan Connection refused porque la máquina destino está viva y su kernel responde con un segmento RST: el SYN del cliente llega, pero no hay ningún proceso escuchando en ese puerto (o en esa interfaz, en el caso de F4). Por eso el error es inmediato. F3 da No route to host porque ni siquiera hay ruta hacia la IP: el kernel del cliente no sabe cómo llegar y falla al intentar enviar el SYN. F5 da

```
socketlab@VM-SERVER:~$ sudo ufw status
[sudo: authenticate] Password:
Status: inactive
socketlab@VM-SERVER:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with
operation (y/n)? y
Firewall is active and enabled on system startup
socketlab@VM-SERVER:~$ sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
22/tcp (v6) ALLOW Anywhere (v6)

socketlab@VM-SERVER:~$ python3 servidor_eco.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] esperando conexiones... (Ctrl+C para terminar)

socketlab@vm-cliente:~$ python3 cliente_eco.py 192.168.56
.10 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] ERROR de red: [Errno 110] Connection timed out
[cliente] socket cerrado
socketlab@vm-cliente:~$
```

Figura 22. F5: [Errno 110] Connection timed out con el firewall activo en el servidor.

Connection timed out porque el cortafuegos descarta los SYN en silencio: no llega RST ni nada, y el cliente agota su timeout (unos dos minutos por defecto en Linux). Desde el cliente parece que el servidor está caído, cuando en realidad está corriendo y filtrando paquetes.

IX-D. Respuestas a las preguntas

E6.1 Complete la tabla con el error exacto de cada caso.

Fallo	Error exacto
F1 Servidor no ejecutándose	Connection refused (inmediato)
F2 Puerto equivocado	Connection refused (inmediato)
F3 IP inexistente	[Errno 113] No route to host (inmediato)
F4 Servidor solo en localhost	Connection refused (inmediato, desde la otra VM)
F5 Cortafuegos bloqueando	[Errno 110] Connection timed out (lento)

E6.2 F1 y F3 producen errores distintos. ¿Por qué? ¿Qué le responde la máquina destino en cada caso?

En F1 la máquina destino está viva y responde: su kernel recibe el SYN, ve que nadie escucha en ese puerto, y contesta con un RST; el cliente recibe el RST y falla de inmediato. En F3 no hay destino alcanzable: el kernel del cliente no encuentra ruta hacia esa IP y falla al intentar enviar el SYN, sin que exista respuesta alguna. La diferencia clave es que en F1 hay un destino que responde con RST y en F3 no hay destino ni respuesta.

E6.3 F1 y F5 se parecen desde la perspectiva del cliente, pero no son lo mismo. ¿Qué comando ejecutaría en el servidor para distinguirlos con certeza?

`sudo ufw status`: si el firewall está activo y bloquea el 5000, es F5; si está inactivo, es F1. Complementa con `ss -ltnp | grep :5000`: si el socket aparece escuchando, el servidor sí corre y el problema es el firewall (F5); si no aparece, el servidor no corre (F1). Combinando ambos comandos se distingue con certeza.

E6.4 F4 es el error más común en laboratorios de redes. Explique, usando lo aprendido en E2, por qué el servidor

“sí está corriendo” y sin embargo es inalcanzable desde la otra VM.

En E2 se vio que el socket de escucha queda atado a la dirección del bind(): con "0.0.0.0" acepta por cualquier interfaz, y por eso el servidor era alcanzable. En F4 se cambió a HOST = "127.0.0.1", con lo cual el socket queda asociado solo a la interfaz de loopback, que sirve para que la máquina se hable a sí misma. La VM-CLIENTE llega por la interfaz Host-only (192.168.56.10), donde no hay ningún socket escuchando, y el kernel responde con RST. Desde el cliente el error es indistinguible de F1, pero el proceso está corriendo: simplemente no acepta conexiones por la interfaz correcta. Por eso el laboratorio insiste en usar 0.0.0.0.

E6.5 Construya una tabla de diagnóstico de tres capas: para cada síntoma, qué capa está fallando y qué comando lo confirma.

Síntoma	Capa que falla	Comando que lo confirma
ping no responde	Red	ping -c 4 192.168.56.10 desde el cliente
ping funciona pero nc da refused	Transporte	ss -ltnp en el servidor
ping funciona y nc da timeout	Transporte (fire-wall)	sudo ufw status en el servidor
nc da success pero el cliente falla	Aplicación	Revisar el código del cliente y del servidor
nc da No route to host	Red	ip -br addr en ambas VMs

El orden importa: siempre de abajo hacia arriba, red, luego transporte, luego aplicación. Si el ping falla, no tiene sentido probar el transporte; si el ping funciona pero el transporte falla, el problema está en el puerto, el proceso o el firewall. Ese es el método de diagnóstico por capas que se usa en producción.

X. R1 REGISTRO DE CONEXIONES

X-A. Objetivo e implementación

El reto consistía en que el servidor, además de hacer eco, registrara en bitacora.log una línea por cada conexión atendida, con marca de tiempo, IP y puerto del cliente, número de mensajes, bytes recibidos y enviados, y duración de la conexión. Este último campo no lo pide el laboratorio, pero da una idea más completa de la actividad del servidor.

Se creó servidor_eco_r1.py a partir del original, agregando un contador por conexión y una función registrar_conexion() que se llama al final de cada sesión:

```
def registrar_conexion(ip_cliente, puerto_cliente,
                      num_mensajes,
                      bytes_recibidos,
                      bytes_enviados, duracion):
```

```
marca = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
linea = (
    f"{marca} | "
    f"cliente={ip_cliente}:{puerto_cliente} | "
    f"mensajes={num_mensajes} | "
    f"bytes_recibidos={bytes_recibidos} | "
    f"bytes_enviados={bytes_enviados} | "
    f"duracion={duracion:.2f}s\n"
)
with open(ARCHIVO_BITACORA, "a", encoding="utf-8") as archivo:
    archivo.write(linea)
```

Dos decisiones valen la pena justificar. El archivo se abre en modo .a" y no "w", porque con "w" cada conexión borraría la bitácora y solo quedaría la última línea. Y la IP y puerto del cliente se obtienen con getpeername(), la forma estándar de consultar el extremo remoto de una conexión ya establecida, como pide el laboratorio.

X-B. Verificación y reflexión

Con dos clientes consecutivos, el archivo acumuló dos líneas:

```
2026-10-05 23:16:14 | cliente=192.168.56.11:60978 |
mensajes=3 | bytes_recibidos=10 | bytes_enviados=
10 | duracion=20.23s
2026-10-05 23:17:09 | cliente=192.168.56.11:35574 |
mensajes=3 | bytes_recibidos=6 | bytes_enviados=
6 | duracion=3.69s
```

La IP y puerto coinciden con lo mostrado por el cliente, los contadores de mensajes y bytes son consistentes con lo intercambiado, y el archivo acumula en lugar de sobrescribirse.

```
socketlab@VM-SERVER:~$ cat bitacora.log
2026-10-06 11:36:58 | cliente=192.168.56.11:58156 | mensajes=2 | bytes_recibidos=9 | bytes_enviados=9 | duracion=5.21s
socketlab@VM-SERVER:~$ python3 servidor_eco_r1.py
[servidor] socket de escucha en ('0.0.0.0', 5800)
[servidor] bitacora: bitacora.log
[servidor] esperando conexiones... (Ctrl+C para terminar)
[servidor] --- conexión aceptada ---
[servidor] cliente remoto : 192.168.56.11:46324
[servidor] extremo local : ('192.168.56.10', 5800)
[servidor] extremo remoto : ('192.168.56.11', 46324)
[servidor] recibidos 7 bytes: b'segunda'
[servidor] enviados 7 bytes: b'SEGUNDA'
[servidor] recibidos 7 bytes: b'entrada'
[servidor] enviados 7 bytes: b'ENTRADA'
[servidor] recv() devolvió 0 bytes -> el cliente cerró la conexión
[servidor] bitacora actualizada: 2 mensajes, 14 bytes recibidos, 14 bytes enviados, 6.79s
[servidor] conexión cerrada. Vuelvo a accept()
^C
[servidor] interrumpido por el usuario
[servidor] socket de escucha cerrado
socketlab@VM-SERVER:~$ cat bitacora.log
2026-10-06 11:36:58 | cliente=192.168.56.11:58156 | mensajes=2 | bytes_recibidos=9 | bytes_enviados=9 | duracion=5.21s
2026-10-06 11:37:22 | cliente=192.168.56.11:46324 | mensajes=2 | bytes_recibidos=14 | bytes_enviados=14 | duracion=6.79s
socketlab@VM-SERVER:~$
```

Figura 23. Contenido de bitacora.log después de varias conexiones, mostrando que el archivo acumula líneas.

Este reto toca algo que aparece en todo sistema distribuido real: dejar rastro de lo que pasó. Los servidores de producción llevan bitácoras para diagnóstico, auditoría y análisis, normalmente enviadas a sistemas centralizados como ELK o CloudWatch. El principio aquí es el mismo, solo que en versión mínima: sin ese rastro, diagnosticar un problema después de que ocurrió es prácticamente imposible.

XI. R2 PROTOCOLO CON FRONTERAS DE MENSAJE

XI-A. Objetivo e implementación

R2 ataca directamente el problema descubierto en E4: TCP no conserva las fronteras entre mensajes. La solución es delimitar cada mensaje con \n y acumular bytes en un buffer hasta encontrar el delimitador.

En el servidor, un `recv()` ya no se trata como un mensaje completo: los bytes se acumulan y solo se procesan los mensajes delimitados:

```
buffer = b"" # fuera del bucle
while True:
    datos = conexion.recv(TAM_BUFFER)
    if not datos:
        break
    buffer += datos
    while DELIMITADOR in buffer:
        mensaje, buffer = buffer.split(DELIMITADOR,
1)
        respuesta = mensaje.upper() + DELIMITADOR
        conexion.sendall(respuesta)
```

Tres detalles de este fragmento son los que hacen que funcione. El buffer se declara fuera del bucle: si se declarara dentro, se borraría en cada iteración y se perdería el fragmento parcial del ciclo anterior (el error más común al implementar este reto, y produce un servidor que pasa pruebas suaves pero falla de forma intermitente). Se usa `while` y no `if` para el delimitador: un solo `recv()` puede traer varios mensajes completos, como demostró E4, y el `while` procesa todos los que haya en el buffer. Y el residuo parcial se conserva: lo que queda tras procesar los mensajes completos es un mensaje incompleto que se guarda para el próximo `recv()`, cuando se complete.

El cliente (`cliente_eco_r2.py`) agrega `\n` a cada envío y usa el mismo patrón de buffer para leer: espera hasta que el buffer contenga un delimitador, y solo entonces separa y muestra la respuesta.

XI-B. Verificación

Se probó con una variante del cliente ráfaga que envía los tres mensajes con delimitador y sin pausa. El cliente recibió UNO, DOS y TRES por separado, y el servidor procesó los tres mensajes uno por uno aunque llegaron sin pausa:

```
[servidor] mensaje completo: b'uno'
[servidor] respuesta: b'UNO\n'
[servidor] mensaje completo: b'dos'
[servidor] respuesta: b'DOS\n'
[servidor] mensaje completo: b'tres'
[servidor] respuesta: b'TRES\n'
```

```
socketlab@server:~$ python3 servidor_eco_r2.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] protocolo: mensajes delimitados por '\n'
[servidor] esperando conexiones... (Ctrl-C para terminar)

[servidor] ---- conexion aceptada ----
[servidor] cliente remoto : 192.168.56.11:54086
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 54086)
[servidor] mensaje completo: b'uno'
[servidor] respuesta: b'UNO\n'
[servidor] mensaje completo: b'dos'
[servidor] respuesta: b'DOS\n'
[servidor] mensaje completo: b'tres'
[servidor] respuesta: b'TRES\n'
[servidor] recv() devolvió 0 bytes -> el cliente cerró la
[servidor] conexion
[servidor] conexion cerrada. Vuelvo a accept()
```

Figura 24. Ejecución de la prueba de ráfaga del cliente y servidor R2, mostrando los tres send y recv del cliente y el procesamiento separado de los mensajes uno, dos y tres por parte del servidor.

Durante las pruebas se ejecutó por descuido el `cliente_rafaga.py` original, sin delimitadores, contra el servidor R2. El servidor aceptó la conexión pero no procesó

nada, porque esperaba un `\n` que nunca llegó, y la conexión terminó sin error visible de ningún lado. Un protocolo es un contrato entre dos partes; si un extremo no lo cumple, la comunicación falla en silencio, lo cual es más peligroso que un error explícito. Por eso el laboratorio pide modificar cliente y servidor juntos.

```
socketlab@server:~$ python3 servidor_eco_r2.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] protocolo: mensajes delimitados por '\n'
[servidor] esperando conexiones... (Ctrl-C para terminar)

[servidor] ---- conexion aceptada ----
[servidor] cliente remoto : 192.168.56.11:48326
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 48326)

socketlab@cliente:~$ python3 cliente_rafaga.py 192.168
.56.10 5000 0
[rafaga] conectado desde ('192.168.56.11', 48326) hacia ('
192.168.56.10', 5000)
[rafaga] pausa entre envíos: 0.0 s

[servidor] mensaje completo: b'uno'
[servidor] respuesta: b'UNO\n'
[servidor] mensaje completo: b'dos'
[servidor] respuesta: b'DOS\n'
[servidor] mensaje completo: b'tres'
[servidor] respuesta: b'TRES\n'
[servidor] recv() devolvió 0 bytes -> el cliente cerró la
[servidor] conexion
[servidor] conexion cerrada. Vuelvo a accept()
```

Figura 25. El `cliente_rafaga.py` original contra el servidor R2: el servidor no procesa nada porque el cliente no envía delimitadores.

R2 aplica de forma directa lo aprendido en E4. De las cuatro estrategias de E4.5, aquí se usaron delimitadores por ser la más simple y suficiente para texto; en producción es más común el prefijo de longitud, como hace HTTP con `Content-Length`, porque funciona con contenido binario y tamaños variables. El principio es el mismo en ambos casos: las fronteras de los mensajes son responsabilidad de la capa de aplicación.

XII. R3 COMANDOS EN LUGAR DE ECO

XII-A. Objetivo e implementación

R3 sustituye el eco por un protocolo de aplicación con semántica propia. Se crearon `servidor_comandos.py` y `cliente_comandos.py`, reutilizando el protocolo delimitado de R2 y agregando una función que decide la respuesta según el comando recibido:

Comando	Respuesta del servidor
HORA	Hora actual del servidor
ECO <texto>	El texto tal cual, sin modificaciones
ESTADISTICAS	Número de mensajes atendidos desde el arranque
ADIOS	Cierra la conexión
Cualquier otro	ERROR comando desconocido

Un detalle de implementación: el contador se pasó como una lista de un solo elemento (`contador = [0]`), porque en Python los enteros son inmutables y no se pueden modificar desde dentro de una función; una lista sí es mutable. El cliente envía los comandos línea por línea y reutiliza el patrón de buffer de R2.

XII-B. Verificación

La sesión de prueba recorrió todos los comandos.

HORA devolvió la hora con el formato esperado; ECO conservó el texto completo, espacios incluidos; ESTADISTICAS mostró el contador acumulado, que crecía con cada comando; se verificó el error de comando desconocido; y ADIOS cerró limpiamente por ambos lados.

Con R3, el servidor deja de ser un eco y pasa a tener un protocolo de aplicación, que es lo que hacen los protocolos reales: HTTP, SMTP, FTP y DNS definen un conjunto de

```
socketlab@VM-SERVER:~$ python3 servidor_comandos.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] comandos: HORA, ECO <texto>, ESTADISTICAS, ADIO
S
[servidor] esperando conexiones... (Ctrl+C para terminar)

[servidor] conexion desde 192.168.56.11:57588
[servidor] comando: b'Hora'
[servidor] respuesta: b'ERROR comando desconocido'
[servidor] comando: b'HORA'
[servidor] respuesta: b'HORA 2026-10-06 11:42:40'
[servidor] comando: b'ECO hola'
[servidor] respuesta: b'ECO hola'
[servidor] comando: b'ESTADISTICAS'
[servidor] respuesta: b'ESTADISTICAS mensajes=4'
```

Figura 26. Salida del servidor R3 procesando los comandos y mostrando las respuestas correspondientes.

comandos con semántica y un formato de respuesta. También aparece un principio de diseño para más adelante: el contador de ESTADISTICAS es un recurso compartido que se modifica con cada comando. Mientras el servidor atiende a un cliente a la vez no hay problema, pero con atención concurrente por hilos habría que protegerlo con un lock para que dos hilos no lo modifiquen a la vez.

XIII. R4 ATENCIÓN A VARIOS CLIENTES

XIII-A. Objetivo e implementación

El servidor construido hasta aquí atiende a un cliente a la vez: mientras procesa los mensajes de uno, no acepta conexiones nuevas. R4 pide resolver eso lanzando un hilo por cada socket que devuelve `accept()`, de modo que varios clientes sean atendidos en paralelo.

Se creó `servidor_threading.py` partiendo del servidor de comandos de R3. El bucle principal ahora lanza un hilo por cada conexión aceptada, y todo el procesamiento se movió a una función `atender_cliente()` que se ejecuta en ese hilo. El hilo se lanza con `daemon=True`, de modo que se cierra automáticamente cuando el programa principal termina.

XIII-B. El problema del contador compartido

El laboratorio advierte que, si se combina R3 con R4, el contador de ESTADISTICAS queda compartido entre hilos. El problema es clásico: la operación `contador[0] += 1` parece atómica, pero internamente son tres pasos: leer el valor, sumarle uno y escribirlo. Si dos hilos ejecutan esos tres pasos intercalados, puede ocurrir que ambos lean el mismo valor, ambos sumen uno, y ambos escriban el mismo resultado: se hicieron dos incrementos pero el contador subió solo uno. Esa pérdida de cuentas se llama *race condition* o condición de carrera, y es el tipo de error más difícil de depurar en sistemas concurrentes, porque no se reproduce de forma consistente.

La solución fue proteger el incremento con un lock: `with lock: contador[0] += 1`. Dentro de ese bloque solo un hilo a la vez puede ejecutar las tres operaciones, y los demás esperan a que el lock se libere. Así el incremento se vuelve atómico y no se pierden cuentas.

XIII-C. Verificación

Se abrieron tres sesiones SSH simultáneas en la VM-CLIENTE y se lanzó el cliente de comandos en cada una, enviando `ECO cliente` y `ESTADISTICAS`. El servidor mostró los tres hilos trabajando en paralelo:

```
[hilo Thread-1 (atender_cliente)] atendiendo a
192.168.56.11:58114
[hilo Thread-2 (atender_cliente)] atendiendo a
192.168.56.11:44614
[hilo Thread-3 (atender_cliente)] atendiendo a
192.168.56.11:44624
[hilo Thread-3 (atender_cliente)] b'ECO cliente' ->
b'ECO cliente'
[hilo Thread-1 (atender_cliente)] b'ECO cliente' ->
b'ECO cliente'
[hilo Thread-2 (atender_cliente)] b'ECO cliente' ->
b'ECO cliente'
[hilo Thread-1 (atender_cliente)] b'ESTADISTICAS' ->
b'ESTADISTICAS mensajes=4'
[hilo Thread-2 (atender_cliente)] b'ESTADISTICAS' ->
b'ESTADISTICAS mensajes=5'
[hilo Thread-3 (atender_cliente)] b'ESTADISTICAS' ->
b'ESTADISTICAS mensajes=6'
```

Los tres clientes fueron atendidos en paralelo, sin que ninguno bloqueara a los demás, y el contador reflejó correctamente el total acumulado: cada cliente envió dos comandos, y los valores mostrados corresponden a los seis comandos procesados hasta ese punto, sin cuentas perdidas.

```
socketlab@VM-SERVER:~$ python3 servidor_threading.py
[servidor] socket de escucha en ('0.0.0.0', 5000)
[servidor] modo concurrente (threading)
[servidor] esperando conexiones... (Ctrl+C para terminar)
[hilo Thread-1 (atender_cliente)] atendiendo a 192.168.56.11:52669
[hilo Thread-2 (atender_cliente)] atendiendo a 192.168.56.11:60146
[hilo Thread-3 (atender_cliente)] atendiendo a 192.168.56.11:60152
[hilo Thread-1 (atender_cliente)] b'ECO cliente' -> b'ECO cliente'
[hilo Thread-2 (atender_cliente)] b'ECO cliente' -> b'ECO cliente'
[hilo Thread-3 (atender_cliente)] b'ECO cliente' -> b'ECO cliente'
[hilo Thread-1 (atender_cliente)] b'ESTADISTICAS' -> b'ESTADISTICAS mensajes=4'
[hilo Thread-2 (atender_cliente)] b'ESTADISTICAS' -> b'ESTADISTICAS mensajes=5'
[hilo Thread-3 (atender_cliente)] b'ESTADISTICAS' -> b'ESTADISTICAS mensajes=6'
```

Figura 27. Salida del servidor R4 mostrando los tres hilos Thread-1, Thread-2 y Thread-3 atendiendo clientes en paralelo.

```
socketlab@VM-SERVER:~$ ss -ntlp | grep :5000
ESTAB 0      0      192.168.56.10:5000 192.168.56.11:52660 users:((("python3",pid=4179,fd=4))
ESTAB 0      0      192.168.56.10:5000 192.168.56.11:60152 users:((("python3",pid=4179,fd=6))
ESTAB 0      0      192.168.56.10:5000 192.168.56.11:60146 users:((("python3",pid=4179,fd=5))
```

Figura 28. Tres sesiones SSH de la VM-CLIENTE corriendo el cliente de comandos contra el servidor concurrente.

R4 introduce la concurrencia en un servidor, algo que aparece en prácticamente cualquier sistema real: Nginx, Apache y la mayoría de los servidores modernos usan hilos, procesos o eventos asíncronos para atender múltiples clientes a la vez. La *race condition* del contador es un recordatorio de que el estado compartido entre hilos requiere cuidado explícito.

XIV. R5 MEDICIÓN DE TIEMPO DE IDA Y VUELTA

XIV-A. Objetivo e implementación

R5 pide medir el *round-trip time* (RTT) de cada mensaje, enviando cien mensajes de 10 bytes y cien de 8000 bytes, y presentar el mínimo, el máximo y el promedio de cada serie. Se crearon dos archivos: `servidor_rtt.py`, un servidor de eco simple con protocolo delimitado, y `cliente_rtt.py`, que mide el tiempo entre el envío y la recepción completa de cada respuesta.

Dos detalles de la implementación del cliente merecen mención. Se usa `time.perf_counter()` y no `time.time()`, porque el primero tiene mayor resolución y no se ve afectado por ajustes del reloj del sistema, algo indispensable para medir intervalos de pocos milisegundos. Y el cronómetro se detiene solo cuando la respuesta está completa, reutilizando el patrón de buffer de R2, porque de lo contrario el RTT saldría artificialmente bajo cuando la respuesta llegara fragmentada.

XIV-B. Resultados e interpretación

Un ejemplo de las pruebas dio la tabla de la figura 29. La serie de 8000 bytes resultó, en promedio, 2.246 ms más lenta que la de 10 bytes, un 154 % aproximadamente.

```
socketlab@vm-cliente:~$ python3 cliente_rtt.py 192.168.56.10 5000
[cliente] conectado a 192.168.56.10:5000
[test] enviando 100 mensajes de 10 bytes...
[test] enviando 100 mensajes de 8000 bytes...
=====
==
Serie                Min (ms)    Max (ms)    Prom (ms)
=====
==
10 bytes              0.986      3.329       1.459
8000 bytes            1.579      29.981      3.704
=====
==
Diferencia promedio: 2.246 ms
[cliente] socket cerrado
```

Figura 29. Tabla de resultados de R5 con el mínimo, máximo y promedio de cada serie, y la diferencia promedio entre ellas.

La diferencia no viene de la latencia de red, que es la misma en ambos casos, sino del tiempo de transmisión: enviar y recibir casi ocho kilobytes toma más tiempo que diez bytes. La observación se ve en los máximos, que pasan de 3.329 ms a 29.981 ms, mientras que los mínimos quedan relativamente cercanos (0.986 ms y 1.579 ms), porque el mínimo depende del mejor caso posible y no del tamaño del mensaje. En una red local como la Host-only el efecto es pequeño, porque el ancho de banda es enorme; en una red real la diferencia sería bastante mayor.

R5 introduce un tema central en sistemas distribuidos: el rendimiento no depende solo de la distancia geográfica, sino de todo lo que ocurre en el camino. Medir el RTT es la primera herramienta de diagnóstico cuando algo se siente lento, y en aplicaciones críticas se usan percentiles (p50, p95, p99) en lugar del promedio, porque el promedio puede esconder colas largas que el usuario sí percibe.

XV. RESPUESTAS A LAS PREGUNTAS DE COMPRENSIÓN

XV-A. Bloque 1 Identidad y multiplexación

Pregunta 1. Defina con sus palabras qué es un socket y en qué se diferencia de un puerto.

Un socket es el punto final de una comunicación: la interfaz que el sistema operativo le da a un proceso para enviar y recibir bytes por la red. Un puerto, en cambio, es solo un número entre 0 y 65535 que identifica una aplicación dentro de una máquina. La diferencia está en que el socket es el objeto con el que trabaja el programa se crea, se configura, se usa y se cierra, mientras que el puerto es una propiedad de ese objeto. Cuando el servidor hizo `bind(("0.0.0.0", 5000))`, asoció un socket al puerto 5000: el socket es el objeto, el 5000 es el número.

Pregunta 2. ¿Por qué una conexión TCP se identifica con cuatro valores y no con dos? Dé un ejemplo concreto tomado de E1 donde dos valores no bastarían.

Porque el puerto solo no alcanza cuando varios clientes se conectan al mismo servidor. En E1, tres clientes simultáneos contra el puerto 5000 produjeron tres conexiones con la misma IP y puerto del servidor pero con distinto puerto efímero del cliente. Si la conexión se identificara solo por el puerto del servidor, esas tres serían indistinguibles. Con la cuádrupla completa, cada una es única, que es justo lo que dice el RFC 9293 en su sección 3.4.1.

Pregunta 3. El cliente no hace `bind()`. ¿Significa eso que no tiene puerto? Justifique con la evidencia de su ejecución.

No: el cliente sí tiene puerto, solo que no lo elige él. Al llamar a `connect()`, el sistema operativo le asigna un puerto efímero del rango 32768–60999. La evidencia está en la línea extremo local : ('192.168.56.11', 34567) que el propio cliente imprimió, y en el ss del servidor, donde cada conexión mostró un puerto efímero distinto. El cliente no necesita `bind()` porque no le importa qué puerto usar: con uno libre basta, y el sistema operativo se encarga de asignarlo.

XV-B. Bloque 2 Ciclo de vida de la conexión

Pregunta 4. Ordene y explique las llamadas del servidor y del cliente. ¿Cuáles son bloqueantes y qué significa exactamente “bloqueante” en términos del proceso y del sistema operativo?

En el servidor: `socket()`, `bind()`, `listen()`, `accept()`, `recv()/sendall()`, `close()`. Las únicas bloqueantes son `accept()`, que espera un cliente, y `recv()`, que espera bytes o el cierre del otro lado. En el cliente: `socket()`, `connect()`, `sendall()`, `recv()`, `close()`, y solo `connect()` es bloqueante, porque espera a que se complete el saludo de tres vías.

Que una llamada sea bloqueante significa que el proceso se suspende, sin consumir CPU, hasta que ocurre el evento esperado: el sistema operativo lo marca como dormido y lo saca de la cola de procesos listos, y lo despierta cuando el evento ocurre. Mientras el proceso espera, otros usan la CPU. En el laboratorio se vio directamente: el servidor bloqueado en `accept()` dejaba la terminal quieta y no consumía CPU.

Pregunta 5. ¿Qué ocurre entre el instante en que el cliente ejecuta `connect()` y el instante en que el servidor retorna de `accept()`? ¿Cuánto de eso programó usted?

Ocurre el saludo de tres vías (SYN, SYN-ACK, ACK), tras lo cual la conexión queda establecida en ambos extremos y el kernel del servidor la coloca en la cola de `accept()`. De todo eso, el programador no escribió nada: el intercambio vive dentro de la pila TCP del kernel, y en el código solo se manifiesta como el tiempo que tarda `connect()` en retornar. La API de sockets es una fachada sobre un protocolo que implementa el kernel.

Pregunta 6. ¿Por qué `accept()` devuelve un socket nuevo? ¿Qué se rompería si devolviera el mismo?

Porque el servidor necesita dos sockets: el de escucha, que debe seguir disponible para aceptar nuevas conexiones, y el de la conexión, dedicado a un cliente específico. Si `accept()` devolviera el mismo socket de escucha, el servidor no podría aceptar más conexiones mientras atiende a un cliente, y además ese socket no está diseñado para datos: no tiene buffer de ninguna conexión, como se explicó en E2. La observación lo confirmó: el socket de escucha `0.0.0.0:5000` sigue existiendo tras aceptar la conexión, y el socket nuevo es un objeto distinto.

XV-C. Bloque 3 Semántica del flujo de bytes

Pregunta 7. Explique, con los datos de su ejecución de E4, la afirmación “TCP no conserva las fronteras de los mensajes”.

En E4, el cliente hizo tres `sendall()` con uno, dos y tres (10 bytes), y el servidor hizo solo dos `recv(): b'uno'` (3 bytes) y `b'dostres'` (7 bytes). Nada se perdió ni se desordenó, pero la frontera entre dos y tres desapareció: llegaron pegados en una sola entrega. TCP entrega un flujo continuo de bytes, no una secuencia de mensajes delimitados; la parte “flujo de bytes” de la definición del RFC 9293 es la que explica el fenómeno.

Pregunta 8. TCP garantiza fiabilidad y orden. Enuncie tres cosas que no garantiza y que un programador podría dar por sentadas erróneamente.

Primero, que cada `send()` corresponda a un `recv()` con los mismos bytes: en E4, tres `sendall()` produjeron dos `recv()`. Segundo, que un `recv()` devuelva todo lo que el emisor envió de una vez: devuelve hasta el tamaño del buffer, y a veces hay que llamarlo en bucle. Tercero, que el otro extremo siga vivo: si se apaga de golpe sin enviar FIN, TCP no se entera de inmediato, como se discutió en E3.4.

Pregunta 9. ¿Por qué `sendall()` es preferible a `send()`? ¿Qué hace `sendall()` internamente?

`send()` puede enviar menos bytes de los que se le pasaron: devuelve cuántos envió y hay que reintentar con el resto. `sendall()` hace ese trabajo internamente, llamando a `send()` en un bucle hasta agotar el mensaje. Por eso es la opción preferida para enviar mensajes completos.

Pregunta 10. Si TCP es fiable, ¿por qué el diseño de un protocolo de aplicación sigue necesitando delimitar mensajes? ¿No es redundante?

No es redundante porque cada capa garantiza cosas distintas. TCP garantiza que los bytes llegan completos y en orden, pero no sabe qué es un “mensaje”: para TCP solo hay una secuencia

de bytes. Solo la capa de aplicación conoce la estructura de sus datos, así que solo ella puede marcar las fronteras. HTTP lo hace con `Content-Length`, Redis con prefijos de longitud, WebSocket con frames con cabecera. En el laboratorio, R2 usó delimitadores `\n`, aunque cualquiera de las cuatro estrategias de E4.5 habría servido.

XV-D. Bloque 4 Transferencia a sistemas distribuidos

Pregunta 11. Usted programó `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()` y `recv()`. Cuando se invoca un servicio REST o gRPC, ¿esas llamadas desaparecen o siguen ocurriendo? ¿Quién las ejecuta?

Siguen ocurriendo, solo que las ejecuta el cliente HTTP o gRPC que se use. Una petición como `GET https://api.ejemplo.com/recurso` hace internamente lo mismo que se hizo en el laboratorio: `socket()`, `connect()` al puerto 443, `sendall()` con la petición, `recv()` con la respuesta y `close()`. El programador ya no ve esas llamadas porque están encapsuladas en la biblioteca, pero debajo sigue habiendo sockets TCP; gRPC usa HTTP/2 sobre TCP igualmente. La API de sockets es la base sobre la que se construyen todas las capas de abstracción.

Pregunta 12. Enumere tres problemas que este laboratorio no enfrenta y que sí aparecen en un sistema distribuido real.

Primero, los fallos parciales: aquí si una VM se cae, la otra sigue; en un sistema real, la red puede particionarse, un servicio puede quedar lento sin estar caído, y los protocolos deben manejarlo con reintentos, timeouts e idempotencia. Segundo, la latencia variable: la red Host-only tiene latencia estable, mientras que en producción varía por carga y congestión, y afecta los timeouts y el orden percibido de los eventos. Tercero, la ausencia de un reloj global: los relojes de máquinas distintas no están perfectamente sincronizados, así que los timestamps locales no sirven para ordenar eventos y hacen falta relojes lógicos. Quedan también fuera del alcance la serialización estructurada, la seguridad con TLS y el descubrimiento de servicios.

Pregunta 13. El servidor y el cliente acordaron implícitamente cuatro cosas: dirección, puerto, codificación de caracteres y significado de los mensajes. ¿Cuáles de esos acuerdos están escritos en el código y cuáles solo existen en la cabeza del programador? ¿Qué riesgo implica lo segundo?

La dirección y el puerto están explícitos en el código. La codificación está semi-explicita: se usa `utf-8` en ambos lados, pero si uno cambiara, los caracteres especiales se verían mal. El significado de los mensajes no está escrito en ningún lado: el acuerdo de que un mensaje es texto terminado en `\n` y que el servidor lo convierte a mayúsculas existe solo en la cabeza del programador.

El riesgo es la fragilidad: si alguien cambia el formato en un extremo, la comunicación se desincroniza sin error visible. Se vio en el laboratorio cuando el `cliente_rafaga.py` original no logró comunicarse con el servidor R2 porque uno

esperaba \n y el otro no lo enviaba. Un protocolo no escrito no se puede comunicar a otro equipo, ni versionar, ni depurar.

Pregunta 14. Si tuviera que sustituir TCP por UDP en esta aplicación, ¿qué dejaría de funcionar y qué tendría que implementar usted mismo?

Se perderían todas las garantías: UDP no asegura que los paquetes lleguen ni que lleguen en orden, no tiene control de flujo, no tiene conexión ni saludo de tres vías. Para que la aplicación funcionara, habría que implementar casi todo lo que TCP ya provee: números de secuencia, ACKs, retransmisiones, reordenamiento en el receptor, control de flujo y detección de duplicados. En la práctica, habría que reimplementar TCP sobre UDP, que es exactamente lo que hace QUIC para tener control total del comportamiento. La aplicación del laboratorio se benefició de que TCP ya trae todo esto, y por eso el código pudo ser tan simple.

XVI. DIFICULTADES ENCONTRADAS Y CÓMO SE RESOLVIERON

Durante el laboratorio se presentaron algunas dificultades menores durante la instalación y configuración inicial de las máquinas virtuales. Principalmente estuvieron relacionadas con el manejo de *VirtualBox*, la configuración de las redes y la familiarización con el entorno de las VMs. Estas dudas se resolvieron progresivamente a medida que se adquiría mayor familiaridad con *VirtualBox* y su configuración.

En E4, al ejecutar el cliente de ráfaga con pausa cero, se observó que los mensajes no siempre llegaban al servidor de la misma forma: podían aparecer separados, agrupados parcialmente o recibirse en un único bloque. Esto llevó a revisar el comportamiento de TCP y posteriormente permitió confirmar experimentalmente lo explicado sobre el flujo de bytes y las llamadas a `sendall()` y `recv()`.

En E5 apareció el error `Address already in use` al reiniciar el servidor. Inicialmente, la consulta mediante `ss` no mostraba el estado `TIME_WAIT` en el servidor, lo que generó cierta confusión. La solución práctica fue documentar que el estado pertenece al extremo que cierra y, para evitar el error al reiniciar el servidor, restaurar la línea `SO_REUSEADDR` que se había comentado para el experimento, con lo que el `bind()` pudo ejecutarse de inmediato.

Una vez superadas estas situaciones, el resto de las pruebas se realizó sin mayores inconvenientes. Las principales dificultades estuvieron relacionadas con la configuración inicial del entorno y con la interpretación de algunos comportamientos observados durante las pruebas de TCP.

XVII. CONCLUSIONES

La principal inferencia del laboratorio es que una comunicación TCP funcional depende más de los supuestos de la aplicación que de la complejidad de la API de sockets. Una vez establecida la conectividad, el mecanismo básico es sencillo; lo que determina la confiabilidad del sistema es cómo la aplicación interpreta el flujo recibido, identifica las conexiones y gestiona sus distintos estados.

La ausencia de fronteras de mensaje en TCP tiene una consecuencia directa: la estructura de los datos no puede delegarse al transporte. Si la aplicación necesita intercambiar mensajes independientes, debe definir explícitamente cómo reconocer su inicio y su final. Por eso, un programa que funciona en una prueba particular puede seguir estando mal diseñado si depende de que cada `send()` produzca un `recv()` correspondiente.

De la cuádrupla se infiere también que el puerto que ofrece el servicio es conceptualmente distinto de las conexiones individuales que el servidor mantiene. Esa separación es la que permite atender a varios clientes simultáneamente sin perder la asociación entre cada conexión y su extremo remoto.

Finalmente, el comportamiento observable de una aplicación distribuida no está determinado solo por su código. La configuración de la red, los estados de TCP, el manejo de los sockets y las condiciones de ejecución pueden modificar el resultado de una prueba aparentemente sencilla. Diseñar software de red exige, entonces, considerar explícitamente las propiedades del transporte y no asumir comportamientos que TCP no garantiza.

REFERENCIAS

- [1] IBM Corporation, *Cómo funcionan los sockets*, IBM Documentation. [En línea]. Disponible en: <https://www.ibm.com/docs/es/i/7.5.0?topic=programming-how-sockets-work>
- [2] F. Gutiérrez Isaza, *Laboratorio N.º 1: Comunicación entre procesos mediante sockets TCP*, Universidad Tecnológica de Pereira, periodo 2026-2.